



امنیت داده و شبکه

نیم‌سال دوم ۱۴۰۴-۰۵

دکتر مهدی خرازی و دکتر مرتضی امینی

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

تمرین چهارم

فهرست مسائل

۱		مسئله ۱
۱		پروتکل مبتنی بر رمزنگاری نامتقارن
۱		پروتکل ایجاد کلید جلسه
۳		پروتکل پرداخت PayDroid
۳		توزیع کلید با سرور مرکزی
۵		مسئله ۲
۵		۲.۱ سلسله مراتب کلید در یک سامانه بانکی
۶		۲.۲ راهبردهای ذخیره‌سازی کلید
۶		۲.۳ مشتق‌سازی کلید از راز مشترک
۷		۲.۴ فضای کلید مؤثر
۸		مسئله ۳
۸		۱.۳ حملات علیه TLS
۸		۲.۳ پیکربندی IPsec برای ارتباطات سازمانی
۱۰		مسئله ۴
۱۰		۱.۴ تکامل مدل اعطای مجوز
۱۱		۲.۴ دور زدن کامل سامانه مجوز

پاسخ مسئله‌ی ۱.

پروتکل مبتنی بر رمزنگاری نامتقارن

آ

این پروتکل برای احراز اصالت کلاینت طراحی شده تا کاربر A ثابت کند همان کسی است که ادعا می‌کند.

- پیام اول $(A \rightarrow B : [ID]_A)$: کاربر A شناسه خودش را برای B می‌فرستد تا اعلام حضور کند و بگوید می‌خواهد ارتباط را شروع کند.
- پیام دوم $(B \rightarrow A : N_B)$: کاربر B یک چالش (Challenge) به صورت یک نانس تصادفی (N_B) می‌فرستد. این کار برای جلوگیری از حمله Replay است تا مطمئن شود پاسخ کلاینت زنده و در لحظه است.
- پیام سوم $(A \rightarrow B : E(PrvKey_A, N_B))$: کاربر A نانس دریافتی را با کلید خصوصی خودش رمز (در واقع امضا) می‌کند و پس می‌فرستد.

چون فقط A به کلید خصوصی‌اش $(PrvKey_A)$ دسترسی دارد و نانس هم جدید بوده، B با رمزگشایی این پیام توسط کلید عمومی A و تطبیق آن با N_B مطمئن می‌شود که در همین لحظه دارد با خود واقعی A صحبت می‌کند.

ب

این پروتکل در برابر حمله جعل هویت از طریق سرور واسط آسیب‌پذیر است. مشکل اینجاست که امضای A روی نانس، هیچ وابستگی به هویت مقصد (B) ندارد. اگر A بخواهد همزمان با یک سرور مخرب یا کاربر دیگر به نام C ارتباط برقرار کند، C می‌تواند نانسی که B برایش فرستاده را عیناً به عنوان چالش برای A بفرستد. A آن را امضا می‌کند و به C می‌دهد. حالا C امضای A روی نانس B را دارد؛ آن را به B می‌فرستد و خودش را به جای A جا می‌زند، بدون اینکه کلید خصوصی A را داشته باشد.

پروتکل ایجاد کلید جلسه

آ

- A فکر می‌کند کلید K'_{AB} را با B به اشتراک گذاشته، چون این کلید داخل یک پیام رمز شده با کلید بلندمدت K_{AB} آمده است؛ کلیدی که از نظر A فقط خودش و B آن را بلد هستند.
- B فکر می‌کند کلید K'_{AB} را با A به اشتراک گذاشته، چون در پیام سوم، A توانسته نانسی را که B تعیین کرده بود (N_A) که در پیام دوم داخل متن رمز بود) استخراج کند و حالا آن نانس را با خود کلید جلسه جدید (K'_{AB}) رمز کرده و پس فرستاده است. این یعنی A پیام دوم را باز کرده و کلید جدید را قبول دارد.
- A فکر می‌کند کلید K'_{AB} تازه است، چون خودش در پیام اول نانس N_A را فرستاده بود و در پیام دوم، همان نانس را به صورت رمز شده پس گرفته است. این یعنی پیام دوم قدیمی نیست و در پاسخ به درخواست فعلی او تولید شده است.
- B فکر می‌کند کلید K'_{AB} تازه است، چون خودش نانس N_A را از پیام اول گرفته، کلید جدید را کنارش گذاشته و فرستاده است. نانس N_A برای B نقش شناسه نشست فعلی را دارد. (البته این فرض ضعف اصلی پروتکل است، چون نانس را A تولید کرده نه B).

ب

چون پروتکل متقارن است و امکان اجرای معکوس (شروع از سمت B) وجود دارد، مهاجم C بدون داشتن کلید بلندمدت K_{AB} می‌تواند از خود A به عنوان یک Oracle استفاده کند تا پیام رمز شده‌ی مورد نظر را برایش تولید کند.

کاربر A قصد دارد یک نشست عادی با B شروع کند و نانس N_A را می‌فرستد. مهاجم این پیام را Intercept می‌کند:

$$A \rightarrow C(B) : A, N_A$$

مهاجم پیام بالا را نگه می‌دارد و یک اجرای معکوس (پروتکل رفرش از سمت B) با خود A باز می‌کند و همان نانس N_A را به عنوان چالش به A می‌فرستد تا خودش را جای B جا بزند:

$$C(B) \rightarrow A : B, N_A$$

حالا A در نقش پاسخ‌دهنده به اجرای معکوس، یک کلید تازه مانند K^* تولید کرده و آن را همراه با نانس N_A با کلید K_{AB} رمز می‌کند و می‌فرستد:

$$A \rightarrow C(B) : E(K_{AB}, [N_A, K^*])$$

مهاجم که به این رمز دست پیدا کرده، همین پیام را عیناً به عنوان پیام دوم اجرای اول به A برمی‌گرداند:

$$C(B) \rightarrow A : E(K_{AB}, [N_A, K^*])$$

از نظر A این پیام معتبر است، چون با کلید K_{AB} رمز شده و حاوی همان نانس اولیه خودش (N_A) است. پس فکر می‌کند پیام از طرف B آمده و تاییدیه نهایی را با کلید جدید K^* می‌فرستد:

$$A \rightarrow C(B) : E(K^*, N_A)$$

مهاجم این تاییدیه را هم عینا به اجرای معکوس برمی‌گرداند تا آن نشست را هم تکمیل کند:

$$C(B) \rightarrow A : E(K^*, N_A)$$

در پایان این فرآیند، B اصلا در جریان ارتباط نبوده و روحش هم از این ماجرا خبر ندارد، اما A کاملا فریب خورده و فکر می‌کند کلید تازه K^* را با B به اشتراک گذاشته است، در حالی که این کلید فقط بین A و مهاجم (که نقش آینه را بازی کرده) قرار دارد. دلیل اصلی موفقیت این حمله، مشخص نبودن نقش فرستنده و گیرنده (هویت‌ها) در داخل بدنه پیام رمز شده است.

ج

بهترین کار این است که B هم یک نانس اختصاصی (N_B) تولید کند و در رمزگذاری پیام دوم مشارکت دهد، یا اینکه هویت‌ها صراحتا داخل پیام رمز شده بیایند:

$$B \rightarrow A : E(K_{AB}, [N_A, N_B, K'_{AB}])$$

$$A \rightarrow B : E(K'_{AB}, N_B)$$

اینطوری B هم مطمئن می‌شود پیام سوم در پاسخ به نانس جدید خودش تولید شده است.

پروتکل پرداخت PayDroid

آ

خیر، این پروتکل اصلا امن نیست. مشکل بزرگ این است که پیام حاوی هویت گیرنده (B) و مبلغ (X) به صورت آشکار فرستاده می‌شود و امضای دیجیتال A فقط روی X و n قرار دارد: $E(PrivKey_A, [X, n])$.

مهاجم (مثلا خود Bob یا یک شنودکننده مثل C) پیام آلیس را در شبکه Intercept می‌کند. مهاجم هویت B را در بخش آشکار پیام حذف کرده و شناسه خودش (مثلا C) را قرار می‌دهد:

$$A \rightarrow S : A, C, X, n, Cert_A, E(PrivKey_A, [X, n])$$

وقتی پیام به سرور S می‌رسد، سرور امضا را با کلید عمومی آلیس چک می‌کند. امضا درست است (چون مبلغ X و نانس n تغییر نکرده‌اند). سرور هم بدون اینکه خبر داشته باشد، پول را به حساب C واریز می‌کند!

ب

برای حل این مشکل، باید هویت گیرنده (B) هم حتما داخل پکیج امضا قرار بگیرد تا کسی نتواند مقصد پول را در مسیر عوض کند:

$$A \rightarrow S : A, B, X, n, Cert_A, E(PrivKey_A, [B, X, n])$$

توزیع کلید با سرور مرکزی

آ

- خط ۱ $(A \rightarrow S : A, B)$: کاربر A به سرور اعلام می‌کند که من (A) می‌خواهم با B یک ارتباط امن داشته باشم، لطفاً یک کلید مشترک برای ما بساز.
- خط ۲ $(S \rightarrow A : E(K_A, R), E(K_B, R))$: سرور کلید تصادفی R را تولید می‌کند. یک نسخه از آن را با کلید مخفی A رمز می‌کند تا فقط A بفهمد، و نسخه دیگر را با کلید مخفی B رمز می‌کند و هر دو را به A می‌دهد.
- خط ۳ $(A \rightarrow B : E(K_B, R), E(R, M))$: کاربر A بخش مربوط به B را برایش می‌فرستد تا B هم به کلید R برسد. همزمان پیغام M را با کلید جلسه R رمز کرده و برای B ارسال می‌کند.

ب

هکر قادر است به پیغام M دسترسی پیدا کند. از آنجایی که این پروتکل هیچ مکانیزمی برای احراز اصالت سرور (مانند امضای دیجیتال سرور) و هیچ نانی برای تضمین تازگی ندارد، در برابر جعل هویت سرور و Replay Attack آسیب‌پذیر است.

اگر هکر (C) از قبل یک کلید نشست قدیمی (R_{old}) و پیام رمز شده‌ی آن $(E(R_{old}, M_{old}))$ را به همراه تیکت B (یعنی $E(K_B, R_{old})$) شنود و ذخیره کرده باشد، می‌تواند مراحل زیر را اجرا کند:

کاربر A طبق روال درخواست خود را به سمت سرور می‌فرستد. هکر این پیام را Intercept می‌کند و اجازه نمی‌دهد به سرور واقعی برسد:

$$A \rightarrow C(S) : A, B$$

هکر خودش را جای سرور S جا می‌زند و پیام‌های قدیمی و لو رفته‌ی نشست قبلی را برای A بازتاب می‌دهد. از آنجا که پیام با کلید K_A رمز نشده، هکر می‌تواند یک کلید تصادفی دلخواه خودش $(R_{attacker})$ را با کلید K_A که احتمالاً از قبل با یک حمله دیگر یا نشست قدیمی به دست آورده رمز کند، یا اگر فرض کنیم هکر کلیدهای قدیمی را دارد، پیام زیر را می‌فرستد:

$$C(S) \rightarrow A : E(K_A, R_{attacker}), E(K_B, R_{old})$$

کاربر A کلید $R_{attacker}$ را استخراج کرده، پیام جدید M را با آن رمز می‌کند و به همراه تیکت قدیمی B ارسال می‌کند:

$$A \rightarrow C(B) : E(K_B, R_{old}), E(R_{attacker}, M)$$

هکر پیام $E(R_{attacker}, M)$ را Intercept می‌کند. چون خودش کلید $R_{attacker}$ را تعیین کرده بود، به راحتی آن را رمزگشایی کرده و به متن آشکار پیغام M دسترسی پیدا می‌کند. این نشان می‌دهد که پروتکل بدون استفاده از نانس و احراز اصالت دقیق سرور کاملاً ناامن است.

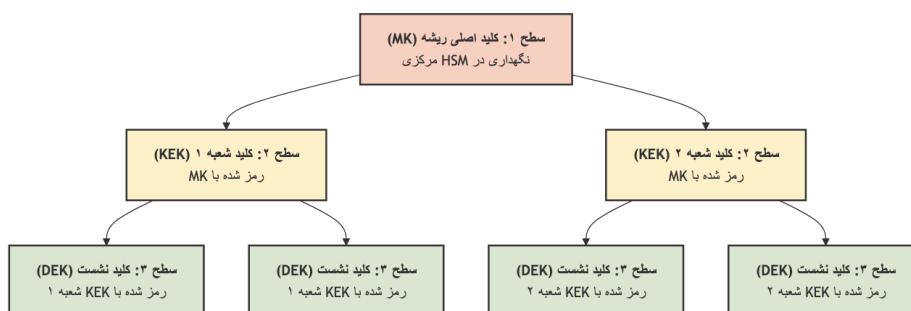
پاسخ مسئله‌ی ۲.

۲.۱ سلسله مراتب کلید در یک سامانه بانکی

آ

ساختار درختی این سامانه به این صورت است که کلید ریشه از همه محافظت می‌کند:

- سطح ریشه (Root): کلید اصلی MK که در داخل ماژول امنیتی سخت‌افزاری شعبه مرکزی تولید و محافظت می‌شود و هرگز به صورت متن آشکار از آن خارج نمی‌شود.
- سطح میانی (Branches): کلیدهای $KEK_1, KEK_2, \dots, KEK_n$ برای شعب مختلف. هر KEK_i توسط MK رمز شده و به صورت ایمن در اختیار شعبه مربوطه قرار می‌گیرد (یعنی ذخیره $(E(MK, KEK_i))$).
- سطح برگ‌ها (Sessions): کلیدهای نشست $DEK_1, DEK_2, \dots$. داده‌های واقعی بانکی با کلید DEK رمز می‌شوند و خود کلید DEK پس از تولید، توسط KEK همان شعبه رمز می‌شود (یعنی $(E(KEK, DEK))$).



ب

از آنجا که این کلید فقط برای یک نشست خاص است، شعاع تخریب بسیار محدود است. تنها داده‌هایی که در همان نشست خاص رد و بدل یا رمز شده‌اند در معرض خطر قرار می‌گیرند. سایر نشست‌های همان شعبه و سایر کاربران کاملاً امن می‌مانند.

ج

شعاع تخریب متوسط تا بزرگ است. اگر KEK یک شعبه لو برود، مهاجم می‌تواند تمام DEK های آن شعبه را رمزگشایی کند. در نتیجه، تمام تراکنش‌ها و نشست‌های مربوط به آن شعبه خاص به مخاطره می‌افتد. با این حال، اطلاعات سایر شعب بانکی و کلید مستر (MK) همچنان امن هستند.

د

- سطح DEK : با فرکانس بالا (معمولاً به ازای هر تراکنش یا اتصال جدید) یک کلید تصادفی جدید تولید شده و با پایان نشست منقضی می‌شود.
- سطح KEK : به صورت دوره‌ای (مثلاً روزانه یا هفتگی). مرکز، یک KEK جدید تولید کرده، آن را با MK رمز می‌کند و برای شعبه می‌فرستد تا از این به بعد نشست‌های جدید با کلید جدید محافظت شوند.

- سطح MK: به ندرت (مثلا سالی یک بار، یا هنگام احساس خطر و ارتقای سخت افزار). تعویض این کلید معمولا نیازمند تشریفات امنیتی موسوم به مراسم کلید (Key Ceremony) با حضور همزمان چند نفر از مدیران امنیتی در کنار دستگاه HSM است.

۲.۲ راهبردهای ذخیره سازی کلید

آ

- محرمانگی بسیار پایین (هرکس به دیسک دسترسی یابد کلید را می دزدد)، در دسترس بودن بسیار بالا (دسترسی سریع و بدون دردسر)، هزینه بسیار پایین (هیچ سربار پردازشی یا مالی ندارد).
- محرمانگی متوسط رو به بالا (وابسته به اینکه KEK کجا نگهداری می شود)، در دسترس بودن بالا، هزینه متوسط (سربار پردازشی برای باز کردن رمز کلید در زمان اجرا).
- محرمانگی بسیار بالا (کلیدها هرگز از سخت افزار استخراج نمی شوند)، در دسترس بودن متوسط/بالا (وابسته به آپتایم و در دسترس بودن شبکه متصل به سخت افزار)، هزینه بسیار بالا (خرید، نگهداری و پیچیدگی های مدیریت دستگاه های گران قیمت).

ب

- این روش فقط در محیط های کاملا ایزوله که هیچ اتصال شبکه ای به بیرون ندارند و تهدید سرقت فیزیکی دیسک یا دسترسی از راه دور وجود ندارد قابل استفاده است (مانند سرورهای ایزوله که کل سیستم عاملشان از Full Disk Encryption بهره می برد و کلید دیسک در بوت لودر تامین می شود).
- برای مقابله با تهدیداتی نظیر سرقت آفلاین دیسک ها، لو رفتن دیتابیس یا بک آپ ها مناسب است. در معماری های رایج وب (مانند الگوی Envelope Encryption) کاربرد دارد، مشروط بر اینکه KEK در یک متغیر محیطی امن یا یک Secret Manager مجزا نگهداری شود.
- برای مدل های تهدید پیشرفته و مهاجمانی که ممکن است بالاترین سطح دسترسی سیستم را به دست آورند کاربرد دارد. در CA های صدور گواهی و هسته مرکزی سامانه های بانکی که قانونا نیاز به تضمین عدم نشت کلید دارند، کاملا الزامی است.

۲.۳ مشتق سازی کلید از راز مشترک

آ

استفاده مستقیم از راز مشترک Z خروجی دیفی-هلمن به عنوان کلید رمزنگاری به سه دلیل مجاز نیست:

۱. خروجی تبادل کلید دیفی-هلمن (Z) لزوما از نظر توزیع آماری بیت ها یکنواخت نیست و ساختار جبری و ریاضی خاصی دارد. در حالی که کلید آماده برای الگوریتمی مثل AES باید کاملا تصادفی و غیر قابل پیش بینی باشد.
۲. ما برای برقراری یک کانال امن به سه مقدار مجزا (کلید رمزنگاری AES، کلید احراز اصالت MAC و بردار مقداردهی اولیه IV) نیاز داریم. بر اساس اصول رمزنگاری، نباید از یک مقدار واحد (Z) برای چندین هدف مختلف استفاده کرد؛ چرا که تعامل این پروتکل ها با یک کلید واحد می تواند منجر به نشت اطلاعات یا حملات متقاطع شود.
۳. کلیدهای نهایی باید به متن و بافتار نشست (مانند نانس های تبادل شده و هویت دو طرف) وابسته باشند. اگر از Z به تنهایی استفاده شود، کلید بین نشست های مختلف یا موازی کاملا مستقل و قاطی نخواهد شد و این موضوع ریسک حملاتی مثل Replay یا تشابه کلید در نشست های موازی را بالا می برد.

ب

بهترین و استانداردترین روش، استفاده از یک تابع مشتق‌ساز کلید بر پایه هش استاندارد است. فرآیند تولید کلیدها به این صورت انجام می‌شود:

۱. مرحله استخراج (Extract): ابتدا آنتروپی راز مشترک Z به کمک یک نمک ($salt$) استخراج می‌شود تا یک کلید شبه‌تصادفی اولیه (PRK) تولید شود:

$$PRK = HKDF - Extract(salt, Z)$$

که در آن برای گره زدن کلید به بافتار نشست فعلی، مقدار $salt$ را از هش کردن نانس‌های طرفین و کل تاریخچه پیام‌های رد و بدل شده ($transcript$) به دست می‌آوریم:

$$salt = H(N_A \parallel N_B \parallel transcript)$$

۲. مرحله بسط (Expand): در این مرحله از روی PRK ، برای هر یک از اهداف مورد نیاز خروجی‌های مجزایی با طول مشخص و با استفاده از برچسب‌های متنی متفاوت و اطلاعات کانتکست استخراج می‌کنیم تا کلیدها از نظر رمزنگاری کاملاً مستقل باشند:

$$K_{enc} = HKDF - Expand(PRK, 'AESkey' \parallel context, ۳۲)$$

$$K_{mac} = HKDF - Expand(PRK, 'HMACkey' \parallel context, ۳۲)$$

$$IV. = HKDF - Expand(PRK, 'IV' \parallel context, ۱۶)$$

به این ترتیب، کلید رمزنگاری (K_{enc})، کلید احراز اصالت (K_{mac}) و بردار مقداره‌ی اولیه ($IV.$) کاملاً تفکیک شده و مستقل از یکدیگر خواهند بود.

برای پیام‌های مختلف در طول نشست نباید از یک IV تکراری استفاده شود؛ بنابراین یا باید برای هر پیام یک IV اختصاصی و جداگانه مشتق شود و یا اینکه از یک Sequence Number یا شمارنده به صورت XOR با IV کمک گرفته شود تا از حملات مربوط به تکرار بردار اولیه جلوگیری به عمل آید.

۲.۴ فضای کلید مؤثر

آ

مهاجم بازه زمانی تولید کلید را می‌داند که معادل ۱ ساعت یا ثانیه است. دقت تابع زمان نیز ۱ میلی‌ثانیه است. از آنجا که Seed این مولد تنها بر اساس زمان است، تعداد Seed های ممکن به صورت زیر محاسبه می‌شود:

$$\frac{۳۶۰۰s}{۰/۰۰۱s} = ۳,۶۰۰,۰۰۰$$

بنابراین فضای کلید مؤثر برابر $۱۰^۶ \times ۳/۶$ حالت است که به توان ۲، رقمی حدوداً برابر با $۲^{۱۷}$ می‌شود.

ب

فضای کلید واقعی و تئوری برای الگوریتم AES-128 برابر با $۲^{۱۲۸}$ حالت است. اما استفاده از زمان سیستم به عنوان Seed، فضای جستجو را از $۲^{۱۲۸}$ به عدد فاجعه‌بار $۲^{۲۲}$ کاهش داده است.

این سیستم کاملاً شکسته است. یک مهاجم اصلاً نیازی به حمله Brute-force روی کل $۲^{۱۲۸}$ حالت ندارد؛ بلکه کافایت ۳ میلیون عدد زمان را به عنوان Seed در اسکرپت خود وارد کرده، کلیدها را تولید و امتحان کند. این کار روی یک سیستم کامپیوتری معمولی در کسری از ثانیه انجام می‌شود. پیامد این اشتباه این است که توسعه‌دهنده به جای استفاده از تولیدکننده‌های امن رمزنگاری مانند ماژول secrets، از توابع ساده random استفاده کرده.

پاسخ مسئله‌ی ۳.

۱.۳ حملات علیه TLS

آ

در پروتکل TLS، مهاجم می‌تواند پیام ClientKeyExchange را رهگیری و تغییر دهد، اما این تلاش برای ربایش با شکست مواجه می‌شود. دلیل آن این است که تمامی پیام‌های مبادله شده در طول Handshake، در انتها توسط پیام Finished احراز اصالت می‌شوند. پیام Finished شامل یک MAC است که روی هش کل تاریخچه پیام‌های قبلی محاسبه می‌شود. از آنجا که مهاجم کلید اصلی و مخفی را ندارد، نمی‌تواند پیام Finished معتبری از جانب کلاینت تولید کند. این دستکاری دقیقاً در آخرین مرحله Handshake، یعنی زمانی که سرور پیام Finished کلاینت را دریافت کرده و هش تاریخچه را با هش محاسبه‌شده توسط خودش مطابقت می‌دهد، آشکار شده و ارتباط فوراً قطع می‌گردد.

ب

۱. مهاجم برای آغاز این حمله باید پیام ClientHello را که در همان ابتدای Handshake از سمت کلاینت فرستاده می‌شود رهگیری کند. او فیلدها و اکستنشن‌های مربوط به نسخه‌های جدید (مثل Supported Versions) را حذف کرده یا تغییر می‌دهد تا سرور تصور کند کلاینت تنها قادر به پشتیبانی از نسخه‌های قدیمی‌تر است.

۲. در صورت موفقیت اولیه، نسخه پروتکل توافق‌شده به یک نسخه قدیمی‌تر (مثل TLS 1.2 یا TLS 1.1) تنزل می‌یابد. در پی آن، پارامترهای امنیتی و Cipher Suites نشست نیز ضعیف‌تر می‌شوند؛ مثلاً به جای تبادل کلید امن با خاصیت Forward Secrecy، ممکن است از RSA استاتیک و الگوریتم‌های هش ضعیف‌تری استفاده شود.

۳. مکانیزم هوشمندانه‌ای برای جلوگیری از این حمله تعبیه شده است. اگر سروری از TLS 1.3 پشتیبانی کند اما به دلیل درخواست کلاینت (که دستکاری شده) مجبور به توافق روی TLS 1.2 شود، یک رشته بایت ثابت و مشخص (Magic String) را در ۸ بایت آخر فیلد ServerHello.Random قرار می‌دهد. کلاینت مدرن (که از TLS 1.3 پشتیبانی می‌کند) پس از دریافت این پیام، آن ۸ بایت را بررسی می‌کند. با دیدن این رشته ثابت، کلاینت متوجه می‌شود که سرور قابلیت نسخه‌های بالاتر را داشته و حمله Downgrade در میانه راه رخ داده است، لذا ارتباط را Abort می‌کند.

۲.۳ پیکربندی IPsec برای ارتباطات سازمانی

آ

۱. استفاده از ESP مناسب‌تر است. از آنجا که ترافیک از اینترنت عمومی عبور می‌کند، برای حفاظت از داده‌های سازمانی علاوه بر احراز اصالت، به محرمانگی نیز نیاز مبرم داریم تا امکان شنود وجود نداشته باشد. پروتکل AH داده‌ها را رمزگذاری نمی‌کند.

۲. باید از Tunnel Mode استفاده شود. در ارتباط بین دو درگاه VPN (مانند روترهای لبه شبکه اصفهان و تهران)، بسته‌های اصلی که حاوی IP‌های نامعتبر داخلی هستند، باید به طور کامل درون یک IP هدر جدید (با آدرس‌های پابلیک درگاه‌ها) کپسوله شوند تا در بستر اینترنت قابل مسیریابی باشند.

۳. ترتیب فیلدها از چپ به راست به این صورت است:

[New IP] | [ESP Header] | [Orig IP] | [TCP/UDP] | [Data] | [ESP Trailer] | [ESP Auth]

بخش‌های رمزگذاری شده: از ابتدای [Orig IP] تا انتهای [ESP Trailer].

بخش‌های احراز اصالت شده: از ابتدای [ESP Header] تا انتهای [ESP Trailer].

ب

۱. پروتکل AH روی کل بسته از جمله بخش‌هایی از IP هدر خارجی (مثل آدرس IP مبدأ) هش محاسبه کرده و آن را احراز اصالت می‌کند. روتر NAT برای عبور دادن ترافیک به اینترنت، آدرس IP مبدأ را به آدرس پابلیک هتل تغییر می‌دهد. این تغییر باعث نامعتبر شدن هش AH در سمت گیرنده شده و بسته دور انداخته می‌شود.
۲. پروتکل ESP در لایه شبکه کار می‌کند و برخلاف TCP/UDP دارای شماره پورت نیست. روترهای NAT (به طور خاص PAT) برای نگاشت و تفکیک ترافیک چندین کاربر داخلی روی یک IP پابلیک، به پورت‌ها وابسته‌اند. بدون پورت، روتر هتل نمی‌داند بسته‌های بازگشتی از سرور شرکت را باید به کدام دستگاه در شبکه داخلی هتل تحویل دهد.
۳. برای رفع این مشکل، از ویژگی NAT Traversal (NAT-T) استفاده می‌شود. NAT-T کل بسته ESP را درون یک هدر استاندارد UDP (معمولاً پورت ۴۵۰۰) کپسوله می‌کند. با اضافه شدن پورت‌های مبدأ و مقصد UDP، روتر NAT به راحتی می‌تواند عملیات ترجمه آدرس را روی بسته انجام دهد.
۴. در این سناریو نیز از Tunnel Mode استفاده می‌شود. زیرا کلاینت (کارمند) یک آدرس IP مجازی متعلق به شبکه داخلی شرکت دریافت می‌کند. بسته‌های تولید شده با این IP مجازی باید در Tunnel Mode درون یک بسته جدید با IP فیزیکی شبکه هتل کپسوله شوند.

ج

۱. این حمله موفقیت‌آمیز نخواهد بود. پروتکل IPsec به صورت ذاتی برای مقابله با ارسال مجدد بسته‌های قانونی محافظت شده است.
۲. این پروتکل از یک Sequence Number همراه با یک پنجره ضد ارسال مجدد (Sliding Window) استفاده می‌کند. فرستنده به هر بسته یک شماره توالی افزایشی و یکتا اختصاص می‌دهد. گیرنده نیز پنجره‌ای از شماره توالی‌های قابل قبول نگه می‌دارد. اگر بسته‌ای دریافت شود که شماره توالی آن تکراری بوده یا از کران پایین پنجره قدیمی‌تر باشد (مانند بسته‌ای که یک ساعت پیش ارسال شده)، بلافاصله آن را به عنوان یک حمله Replay شناسایی کرده و دور می‌ریزد.
۳. اگر Anti-Replay Window به اشتباه غیرفعال شده یا اندازه‌اش آن قدر بزرگ تنظیم شده باشد که توالی‌های یک ساعت پیش هنوز از پنجره خارج نشده باشند، گیرنده بسته تکراری را به عنوان یک بسته معتبر می‌پذیرد. در نتیجه، سیستم بانکی شرکت همان دستور انتقال وجه را مجدداً اجرا کرده و تراکنش بانکی تکرار می‌شود که خسارت مالی سنگینی به همراه خواهد داشت.

پاسخ مسئله‌ی ۴.

۱.۴ تکامل مدل اعطای مجوز

آ

• فهرست نصب (Install-time Manifest):

- ضعف امنیتی: مشکل همه یا هیچ و پذیرش کورکورانه. کاربر برای نصب برنامه مجبور است تمام مجوزها را یکجا تایید کند. از آنجا که کاربران معمولاً متن طولانی مجوزها را نمی‌خوانند، برنامه‌های مخرب به راحتی به همه چیز دسترسی دائمی و پس‌زمینه پیدا می‌کنند.
- ضعف کاربرپذیری: کاربر در لحظه نصب هیچ کانتکستی ندارد که بداند چرا برنامه مثلاً به دوربین نیاز دارد، چون هنوز با محیط برنامه و فیچرهای آن کار نکرده است.

• درخواست لحظه‌ای (Time-of-use Prompt):

- ضعف امنیتی: خستگی از هشدارها. اگر برنامه پشت سر هم پاپ‌آپ نشان دهد، کاربر برای رد شدن از آن بدون دقت تایید را می‌زند. همچنین در مدل‌های اولیه، وقتی کاربر در لحظه تایید را می‌زد، مجوز برای همیشه (حتی در پس‌زمینه) به برنامه داده می‌شد.
- ضعف کاربرپذیری: قطع کردن جریان کار کاربر. اگر پیام درست در لحظه نامناسبی ظاهر شود، باعث آزار کاربر شده و ممکن است بدون فکر آن را رد کند که باعث از کار افتادن ویژگی‌های برنامه می‌شود.

ب

- ضعفی که برطرف شد: مشکل فقدان کانتکست و پذیرش کورکورانه در زمان نصب حل شد. کاربر حالا فقط زمانی پیام را می‌بیند که مثلاً روی دکمه عکس‌گرفتن کلیک کرده است، پس منطق درخواست مجوز را درک می‌کند و دسترسی‌ها به صورت Granular اعطا می‌شوند.
- ضعفی که باقی ماند: مشکل دسترسی دائمی پس از یک بار تایید. اندروید ۶ هنوز تفاوت بین دسترسی موقت و دسترسی دائمی را به خوبی تفکیک نکرده بود؛ برنامه‌ای که یک بار برای یک کار خاص مجوز می‌گرفت، می‌توانست در پس‌زمینه هم به دسترسی ادامه دهد. همچنین خطر خستگی از پاپ‌آپ‌ها به قوت خود باقی ماند.

ج

- مجوز یک‌بار مصرف (Only this time): دقیقاً کاستی دسترسی دائمی پس از تایید را هدف گرفته است. با این کار، برنامه فقط در همان نشست و تا زمانی که باز است مجوز دارد و برای دفعه بعد باید دوباره اجازه بگیرد. این یعنی جلوگیری از سوءاستفاده‌های پس‌زمینه.
- موقعیت تقریبی در برابر دقیق: کاستی بیش‌امتیازی را هدف می‌گیرد. بسیاری از برنامه‌ها (مثل هواشناسی) فقط نیاز دارند بدانند کاربر در کدام شهر است، نه مختصات دقیق GPS. این ویژگی به کاربر حق انتخاب برای اعمال اصل حداقل دسترسی را می‌دهد.
- شفافیت ردیابی برنامه: کاستی ردیابی نامرئی و تبانی شرکت‌های تبلیغاتی را هدف قرار داده است. برنامه‌ها با استفاده از شناسه‌های یکتا (مثل IDFA) داده‌های رفتاری کاربر را به اشتراک می‌گذاشتند. این سازوکار، ردیابی بین برنامه‌ای را از حالت پیش‌فرض پنهان، به حالت شفاف نیازمند تایید (Opt-in) تبدیل کرد.

۲.۴ دور زدن کامل سامانه مجوز

آ

- کانال جانبی: زمانی اتفاق می افتد که یک برنامه به تنهایی و بدون داشتن مجوز یک داده حساس، از طریق یک مسیر یا منبع محافظت نشده (مثلا فراداده‌های یک فایل دیگر) به آن داده دسترسی پیدا می‌کند. سیستم عامل در محافظت از مرزهای داده ناکام بوده است.
- کانال پنهان: زمانی رخ می‌دهد که دو برنامه مجزا روی دستگاه با هم تبانی می‌کنند. برنامه‌ای که دسترسی دارد داده را می‌خواند و از طریق یک منبع مشترک سیستمی ظاهراً بی‌خطر، آن را به برنامه‌ای که مجوز شبکه دارد می‌رساند.

هر دو روش اصل ایزوله‌سازی برنامه‌ها را دور می‌زنند؛ اما کانال جانبی از ضعف خود سیستم در نشت اطلاعات بهره می‌برد، در حالی که کانال پنهان از تبانی اپلیکیشن‌ها و دور زدن مرزهای ارتباطی استفاده می‌کند.

ب

- نمونه اول (استخراج موقعیت از روی EXIF عکس): یک کانال جانبی است. داده هدف موقعیت مکانی بوده که از مسیر فراداده‌های تصاویر گالری به دست آمده است.
- نمونه دوم (تخمین موقعیت از روی BSSID و وای‌فای): یک کانال جانبی است. داده هدف موقعیت مکانی بوده که از مسیر اطلاعات شبکه وای‌فای و شناسه روترها و نگاشت آن‌ها استخراج شده است.
- نمونه سوم (نوشتن IMEI روی حافظه مشترک): یک کانال پنهان (تبانی) است. داده هدف شناسه دائمی دستگاه بوده که توسط برنامه مجاز روی حافظه مشترک نوشته شده تا برنامه بدون مجوز آن را بخواند.

ج

اصلاح پنجره درخواست مجوز فایده‌ای ندارد، چون برنامه اصلاً از آن مسیر عبور نمی‌کند. کانال‌های جانبی از سطح حمله غیرمنتظره استفاده می‌کنند؛ طراح سیستم عامل در نظر نگرفته که مثلاً از روی فراداده عکس، وضعیت شبکه وای‌فای، یا دمای باتری بشود موقعیت یا اطلاعات حساس کاربر را بیرون کشید. چون این مسیرها محافظت شده نبودند، شناسایی و بستن تمام این روزنه‌ها بدون خراب کردن کارکرد عادی سیستم عامل بسیار دشوار است.

د

- پروتکل و سازوکار کدگذاری: منبع مشترک انتخابی را حجم اشغال شده حافظه نهان در نظر می‌گیریم.
- کدگذاری: برنامه A برای فرستادن بیت 1 حجم مشخصی از حافظه نهان را پر می‌کند و برای فرستادن بیت 0 آن را خالی می‌کند. برنامه B با چک کردن حجم اشغال شده، صفر یا یک را می‌خواند.
- همگام‌سازی: زمان را به بازه‌های ۵۰ میلی‌ثانیه‌ای تقسیم می‌کنیم. در ۲۰ میلی‌ثانیه اول، A تغییر را اعمال می‌کند و در ۲۰ میلی‌ثانیه دوم، B وضعیت را می‌خواند.
- نرخ نشت داده و زمان انتقال: با فرض بازه ۵۰ میلی‌ثانیه‌ای، نرخ انتقال ۲۰ بیت بر ثانیه خواهد بود. برای انتقال یک شماره تلفن ۵۰ بیتی، تنها ۲.۵ ثانیه زمان نیاز است.
- عوامل محدودکننده و اهمیت تهدید: در عمل، نویز سیستم (تغییر حجم حافظه توسط سایر برنامه‌ها) و تاخیر زمان‌بندی پردازنده این نرخ را محدود می‌کنند. با این حال، حتی یک نرخ بسیار پایین (مثلاً چند بیت در ثانیه) یک تهدید بسیار جدی است؛ زیرا داده‌های حیاتی مثل رمز عبور، شماره کارت یا IMEI حجم بسیار کمی دارند و به تدریج در خفا به طور کامل لو می‌روند.